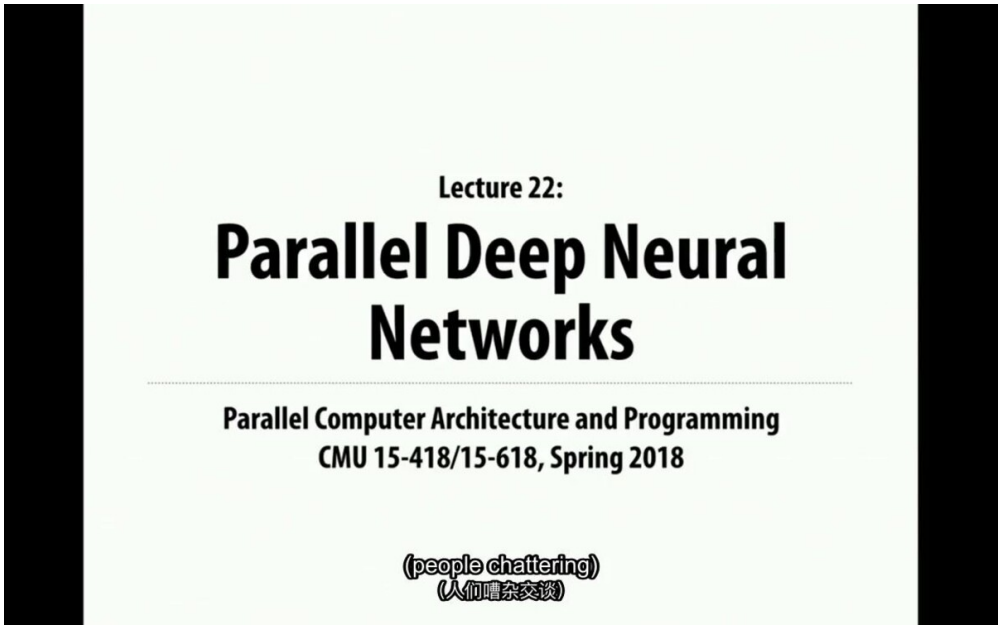
课程笔记

并行深度神经网络：卷积、推理与训练

CMU 15-418/618 并行计算机架构与编程 · Lecture 26（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 17 分 26 秒
视频链接：`https://www.bilibili.com/video/BV1594y167Ue/`

目录

1 深度神经网络：回到结构化数值计算 2
1.1 一场革命 . . . 2
1.2 基本单元 . . . 3
1.3 ReLU：一个极其简单的非线性 . . . 4
1.4 本章小结 . . . 4

2 训练与推理：两个不同的问题 5
2.1 量级差异 . . . 5
2.2 网络拓扑 . . . 6
2.3 连接方式 . . . 6
2.4 本章小结 . . . 6

3 卷积：权重共享带来的三重节省 7
3.1 从图像卷积到卷积层 . . . 7
3.2 滤波器在做什么 . . . 8
3.3 多滤波器 . . . 9
3.4 叠加更多层 . . . 10
3.5 池化 . . . 11
3.6 本章小结 . . . 11

4 推理的计算结构 11
4.1 任务规模 . . . 11
4.2 本章小结 . . . 13

5 推理的计算结构 13
5.1 循环嵌套与局部性 . . . 13
5.2 内存占用 . . . 14
5.3 本章小结 . . . 15

6 训练：梯度下降与损失 15
6.1 本章小结 . . . 15

7 训练：梯度下降与损失 16
7.1 泛化与评估集 . . . 16
7.2 softmax 与损失 . . . 17
7.3 本章小结 . . . 18

8 总结与延伸 18

1　深度神经网络：回到结构化数值计算

这一讲讲并行计算与深度神经网络的关系。讲者开场说这个话题想必很多人感兴趣。

1.1　一场革命

深度神经网络彻底改变了 AI 与机器学习领域，其影响之广难以描述。讲者举了几个例子：Google DeepMind 在围棋上的胜利；图像识别的巨大进步——起初问题只是" 这张图里有猫吗"，现在则变成" 猫在图中哪里"，还能判断它在走、在睡还是在吃，以及是什么品种。语音识别与语言翻译也已经完全交给深度神经网络。

他回忆几年前访问 Google 时，那里的高层说他们基本上把为特定应用开发的所有精巧基础设施都拆掉了，全部换成深度神经网络，用海量数据训练然后放手让它跑。

许多过去需要精细领域知识的问题，现在只需投入大量训练数据。



图 1：训练与评估深度神经网络：近年多项 AI 突破背后的技术[1]

深度学习把 AI 拉回数值计算的地盘

讲者特别指出这一点：深度学习让 AI 重新变成结构化的数值计算问题，看起来更像科学计算而非符号推理。这个转变是它与并行计算天然契合的根本原因——它由大量规则化的乘加运算构成，而不是由不规则的控制流构成。

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

1.2 基本单元

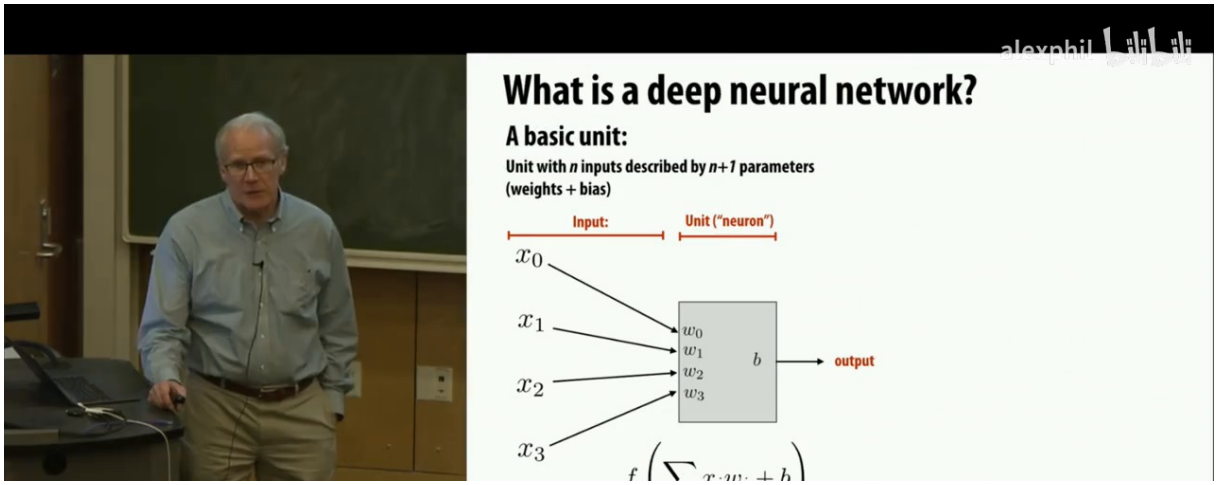


图 2: 深度神经网络的基本单元：n 个输入与 $n+1$ 个参数（权重与偏置）[2]

讲者说明基本想法源自七十年代末甚至更早：一种非常初等的计算，由一组计算元件构成的网络完成。

基本单元是：对若干输入做加权求和。假设这些是实数值，权重也是实数。算出加权和，加上某个偏置，然后对这个结果应用一个非线性函数。

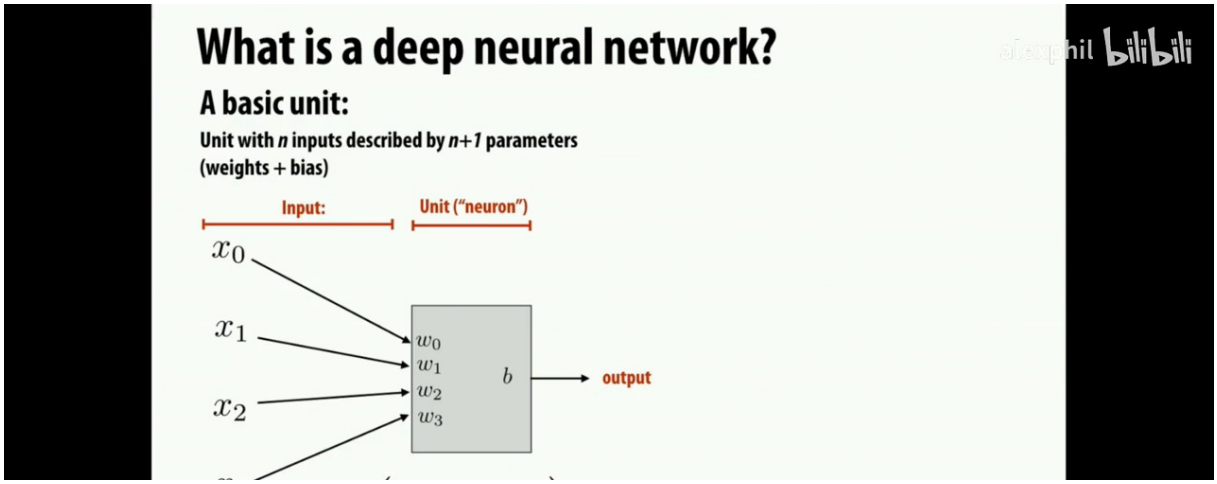


图 3: 激活函数：早期采用饱和型非线性函数[3]

$$y = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- x_i：第 i 个输入值；
- w_i：对应的权重，共 n 个；
- b：偏置，因此该单元共有 $n+1$ 个参数；
- σ：非线性激活函数。

[2] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。
[3] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

1.3 ReLU：一个极其简单的非线性

过去人们设计过各种精巧的函数：输出被限制在零到一之间或某个有界范围，随着输入增大而衰减但不完全消除——不是硬截断，更像是趋近某个极限。

但最近变成了一个简单到离谱的函数：取该值与零的最大值。

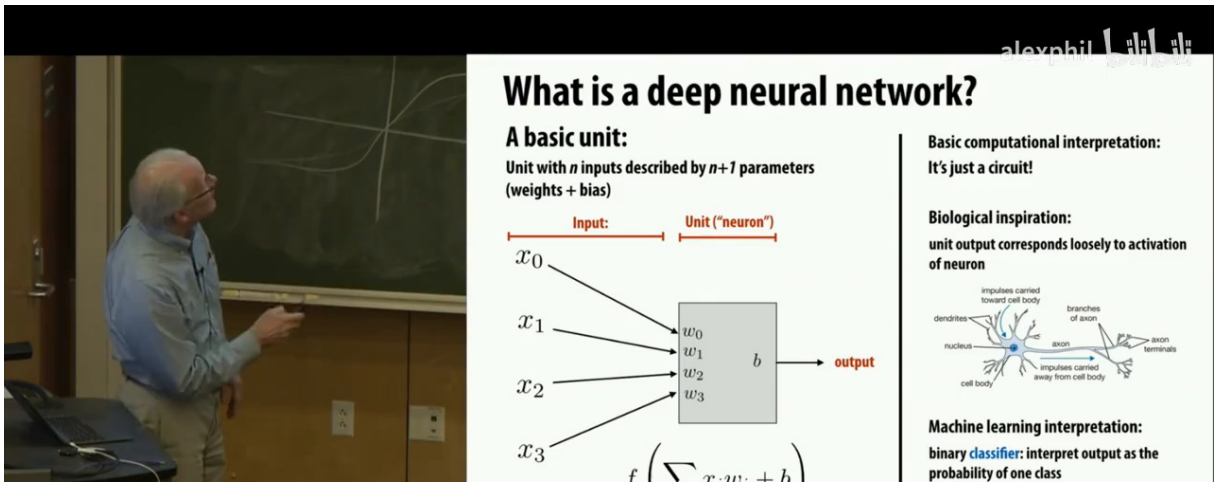


图 4: ReLU：取输入与零的最大值，正区间直通、负区间关闭[4]

$$\mathrm{ReLU}(z) = \max(z, 0)$$

- z：加权和加偏置的结果；
- 若 z 为正，输出就是 z 本身；
- 若 z 为负，输出为零，相当于该单元被”关闭”。

这被称为修正线性单元，也叫铰链函数。它显然是非线性的，但极其简单，而且被证明非常有效。

”脑启发”是设计直觉的来源，不是正确性的依据

神经网络的组件选择确实受生物神经元启发，但它们的有效性由经验结果而非神经科学论证支持。因此调参、换激活函数、改结构都是工程行为：以验证集表现为准，不以”更像大脑”为准。

讲者还做了一个重要澄清：这东西隐约受生物学启发，反映了真实神经元的某些特点；但说它基于某种关于大脑活动的根本理论，则完全是胡说。它是受大脑启发，而非模仿大脑。

1.4 本章小结

深度神经网络彻底改变了 AI，并把问题拉回结构化数值计算。基本单元是加权和加偏置再加非线性激活。早期用饱和型函数，现在普遍用 ReLU——取与零的最大值，简单却有效，且只是受大脑启发而非模仿。

下一章区分两个不同的问题：训练与推理。

[4]视频画面时间区间：00:06:10–00:06:25。若为裁剪图，时间区间仍对应原视频画面。

2　训练与推理：两个不同的问题

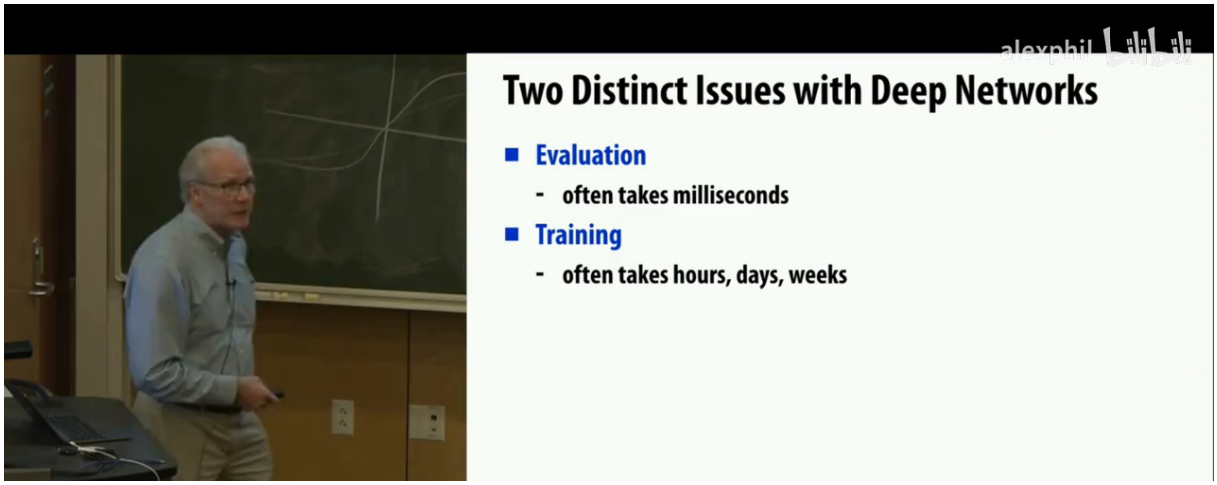


图 5: 深度网络的两个不同问题：推理常以毫秒计，训练常以天计[5]

2.1　量级差异

讲者把这件事拆成两个问题，这个区分决定了后续所有优化方向。

训练：让模型学会模仿观察到的相关性，并且针对目标语言专门训练。这通常需要 7 天以上，长则数周甚至数月，以及巨量算力——Google 的数据中心会动用大量节点连续运行多日，只为把这个模型建起来。

推理：某人往搜索框里输入英文、想翻译成中文，这就是把文本喂进网络、计算所有神经元的值。这相对快速简单，但仍然涉及可能十亿次以上的运算。

推理的算力需求由”调用次数 × 单次代价”决定

单次推理远便宜于一次训练，但推理的调用次数可能比训练多出许多个数量级，而且常在资源受限的设备（手机）上进行。因此总能耗与总延迟的瓶颈往往在推理侧。这解释了为什么移动端神经网络加速是一个独立且重要的研究方向。

[5] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

2.2 网络拓扑

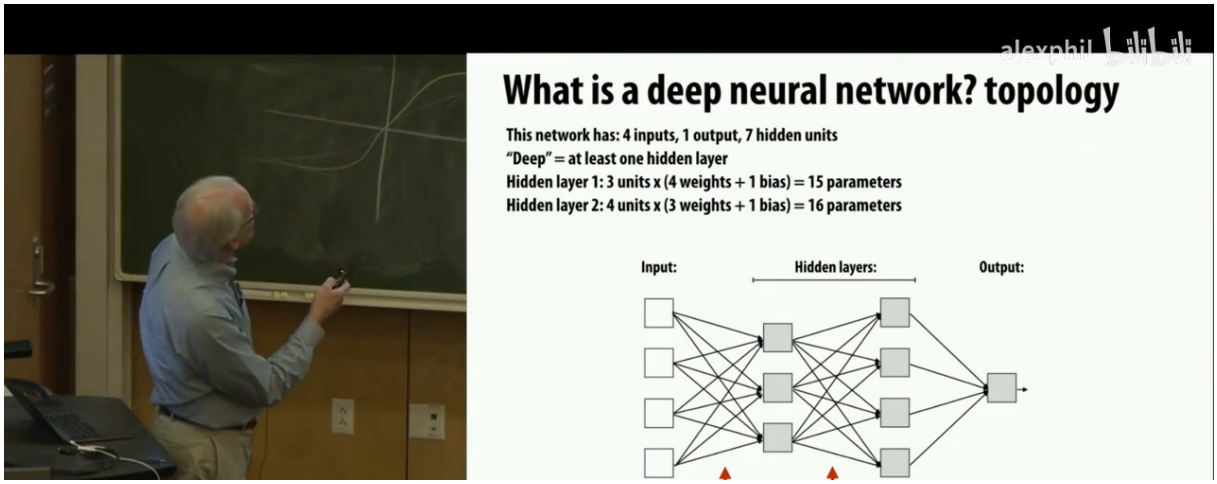


图 6: 网络拓扑：4 个输入、1 个输出、7 个隐藏单元[6]

网络的构成方式是：有一批计算元件或神经单元，按某种结构连成网络。输入是一张图，输出是一组分类器结果。输入与输出之间是隐藏单元。

每个单元都有与其所有输入值对应的一组权重，以及偏置。

讲者说为了讨论推理，先假设这些权重已经确定，稍后再讲怎么得到它们。推理过程就是从输入到输出把这些计算全部做一遍。

他也坦承网络的结构（该有几层、每层多少单元）仍带有相当程度的”黑魔法”成分。

结构选择缺乏理论指导意味着必须靠实验

既然层数与宽度没有可靠的理论推导，网络设计就只能靠经验与搜索。这带来两个后果：一是训练成本高，要试很多配置；二是知识难以迁移，在一个数据集上最优的结构未必在另一个上最优。

2.3 连接方式

讲者区分两类连接。全连接层允许一层所有神经元向下一层所有神经元提供值——这当然相当于完整图，是昂贵的东西，因此相对少见。

但在图像处理中，我们天然有局部性要保留，因此常做稀疏连接。

2.4 本章小结

训练与推理是两个不同的问题：训练需要数天到数月与巨量算力，推理单次快速但调用次数极多且常在受限设备上进行。网络由输入、隐藏单元与输出分类器构成，每个单元有 $n+1$ 个参数。连接方式分全连接与稀疏连接两类。

图像处理中的稀疏连接正是卷积。

[6] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

3 卷积：权重共享带来的三重节省

3.1 从图像卷积到卷积层

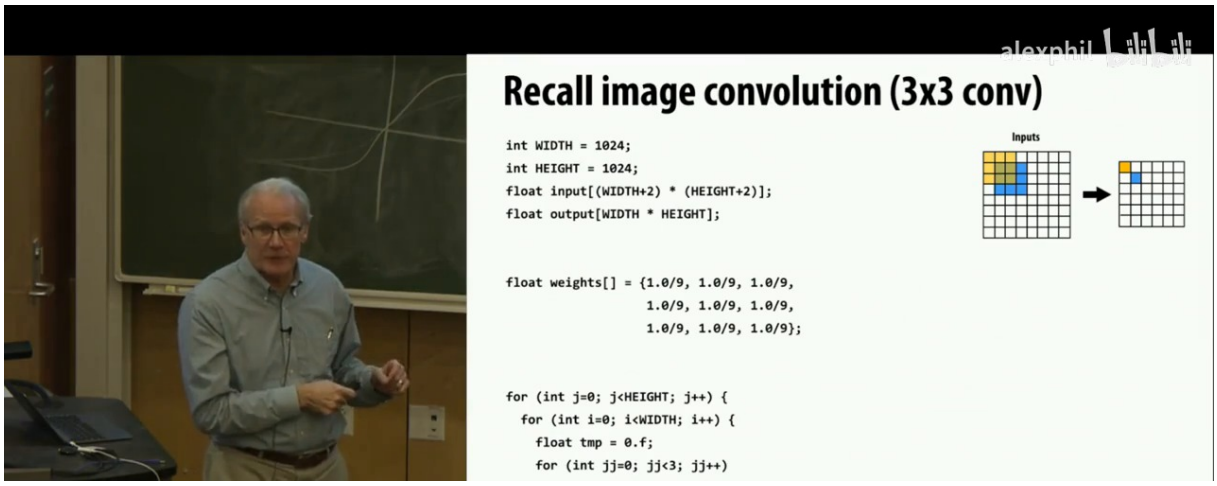


图 7: 回顾图像卷积：1024×1024 输入上的 3×3 卷积[7]

讲者回到图像卷积。此前看过一个简单情形：对某像素最近的九个值使用相同权重。把它推广到任意权重，就很像神经网络的第一部分——对输入做加权和。

卷积的另一个特点是：在图像的每一点都应用同一组权重。

因此有一类网络就叫卷积神经网络，正是这个操作的神经网络版本：有权重，但在给定的连接范围内，权重处处相同——同一个滤波操作应用于整个输入。

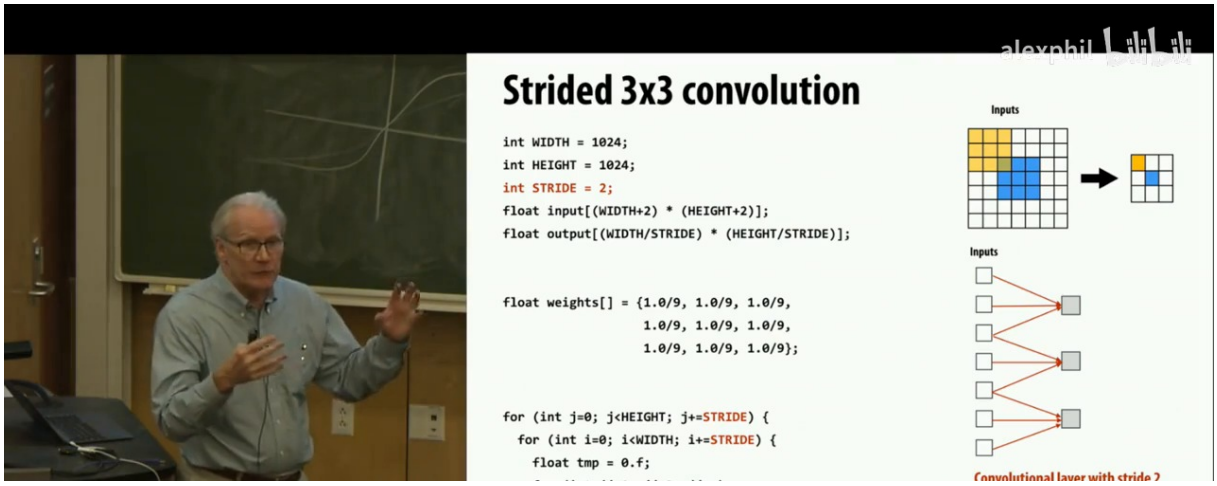


图 8: 带步长的 3×3 卷积：以步长 2 降采样[8]

[7] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

权重共享是卷积层的核心收益

全连接层每个连接都有独立权重，参数量随输入输出规模乘积增长；卷积层只保留一个滤波器的参数，与图像尺寸无关。因此卷积层更易训练（参数少）、更易存储（模型小）、更易推理（计算少）。讲者的结论很直接：能卷积时就优先卷积。

3.2 滤波器在做什么

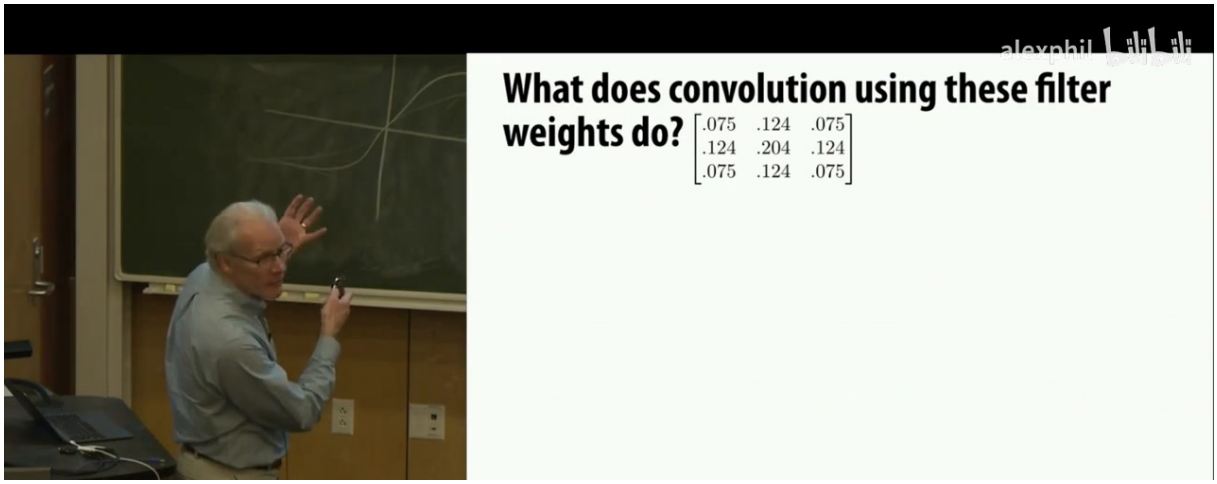


图 9: 这组滤波器权重会做什么：中心权重最高、向外递减[9]

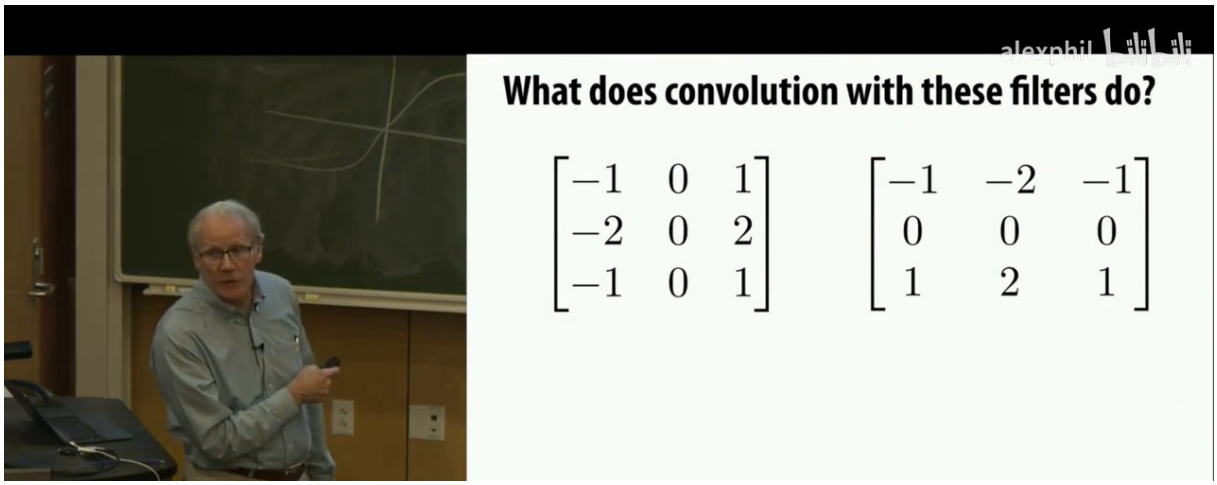


图 10: 高斯模糊：权重按半径呈高斯衰减[10]

讲者展示了一组权重：被保留的像素权重最高，随着距离增加权重递减，按半径呈某种函数关系下降。这实际上是对高斯模糊的近似——权重随半径呈高斯衰减。这也是往图片里引入模糊的常见做法。

[9] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

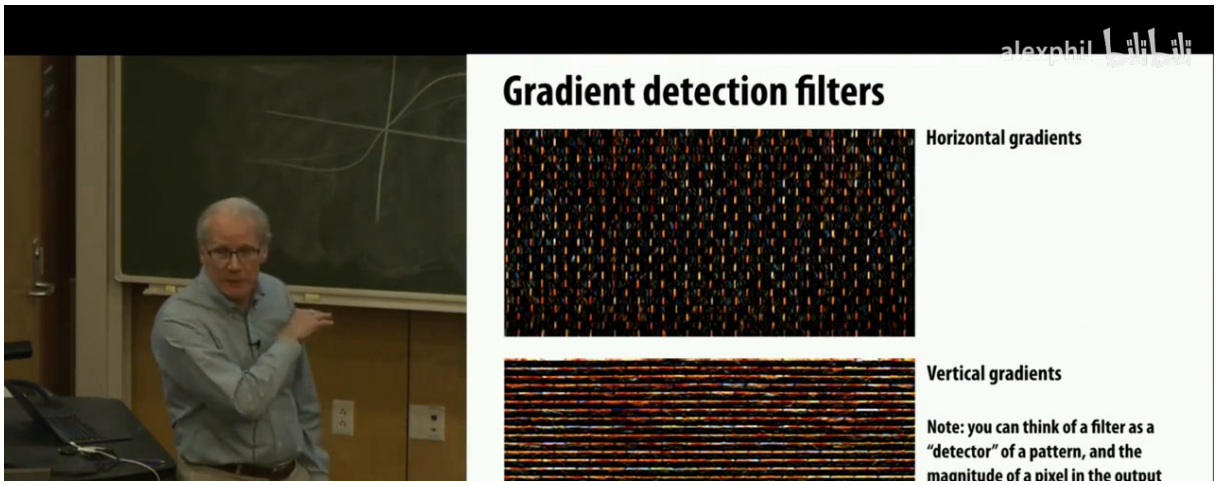


图 11: 梯度检测滤波器：水平与垂直边缘[11]

滤波器权重可以对应到经典图像处理算子

高斯模糊对应权重随半径递减；边缘检测对应正负交替的差分权重。这说明卷积层并非全新的数学，而是把经典图像滤波的算子族参数化了——网络要学的是”该用哪些算子、以什么顺序组合”。

他接着展示含负值的滤波器。这类是做边缘检测的——本质上在求导数，可以想成 x 或 y 方向的空间导数，也就是梯度。

3.3 多滤波器

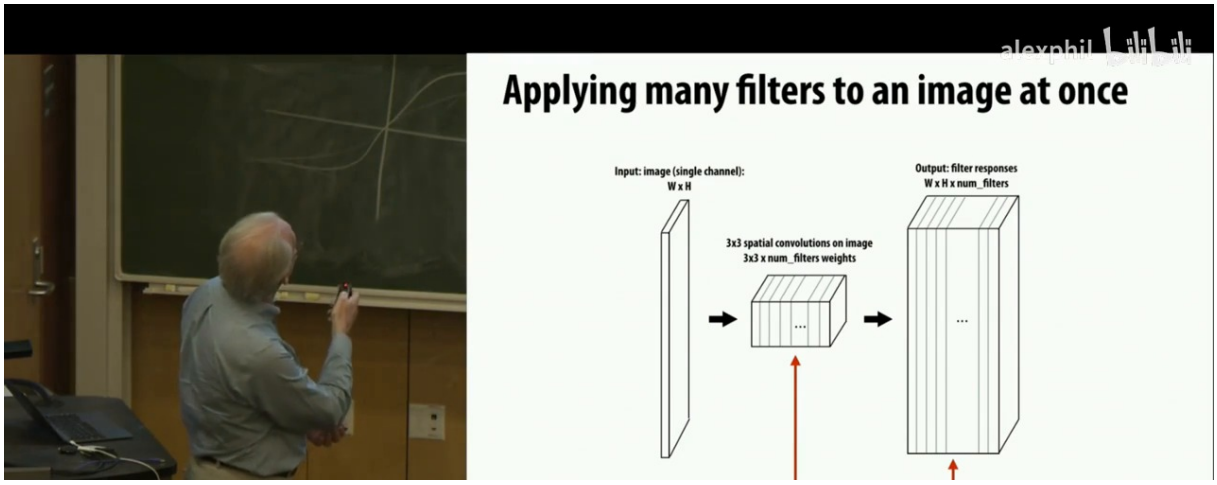


图 12: 同时应用多个滤波器：k 个独立计算产生 k 个响应[12]

讲者在此澄清一个容易误解的图示：我们常想把同类型的多个滤波器应用于一张图，但使用不同的权重。图中画成一摞，但那不表示这些块之间在传递信息。

[11] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

实际上每一个都是独立的滤波操作，各自有自己的权重，各自产生自己的输出。所以看到这个堆叠时，不要以为它们之间有信息流动——那只是画法，不是计算模型。

3.4 叠加更多层

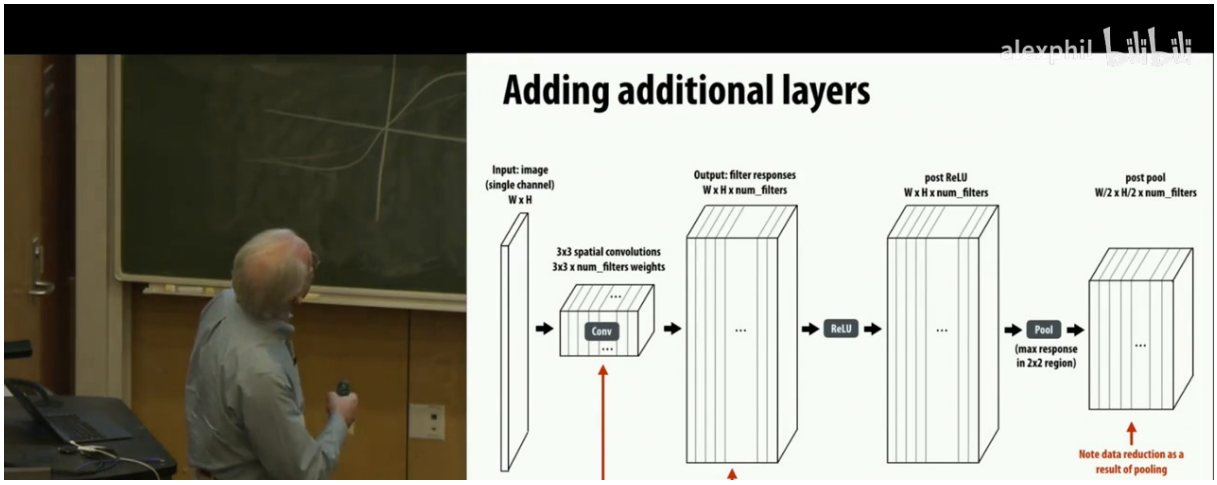


图 13: 叠加更多层：卷积、ReLU 与池化的序列[13]

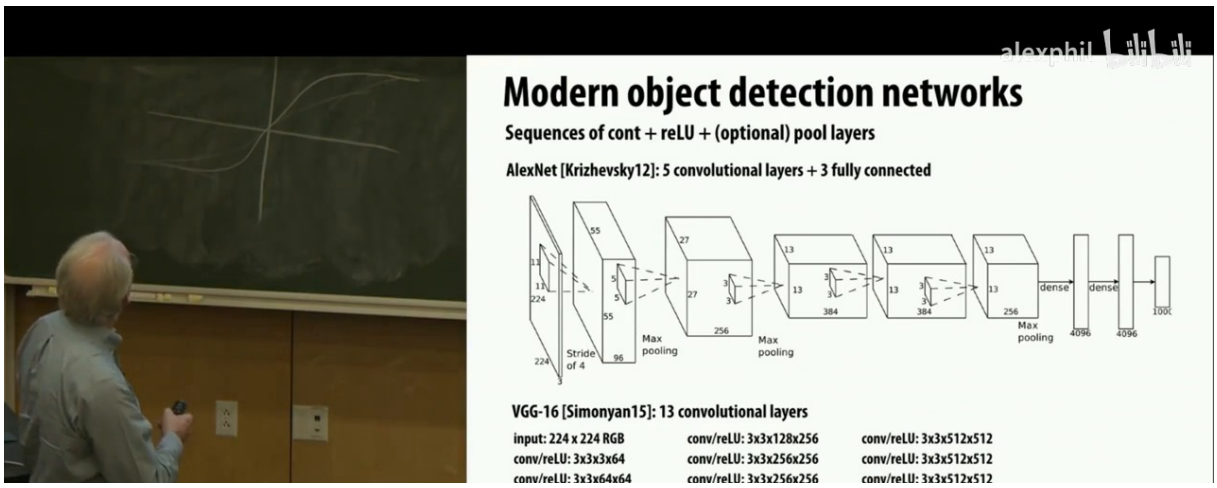


图 14: 现代目标检测网络：卷积 + ReLU + 可选池化的序列（AlexNet）[14]

讲者说明为什么必须有非线性：如果只用线性函数，得不到有趣的结果。这是最早的尝试者（他们把这些叫感知机）遇到的困境——他们不明白为什么什么有意思的事都没发生，原因就是纯线性函数不够，系统必须有非线性才有效。

因此所有滤波响应都要过一个非线性函数。此时这些东西的尺寸仍与原始图像相同，我们可能想把它缩小。

[13] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

[14] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

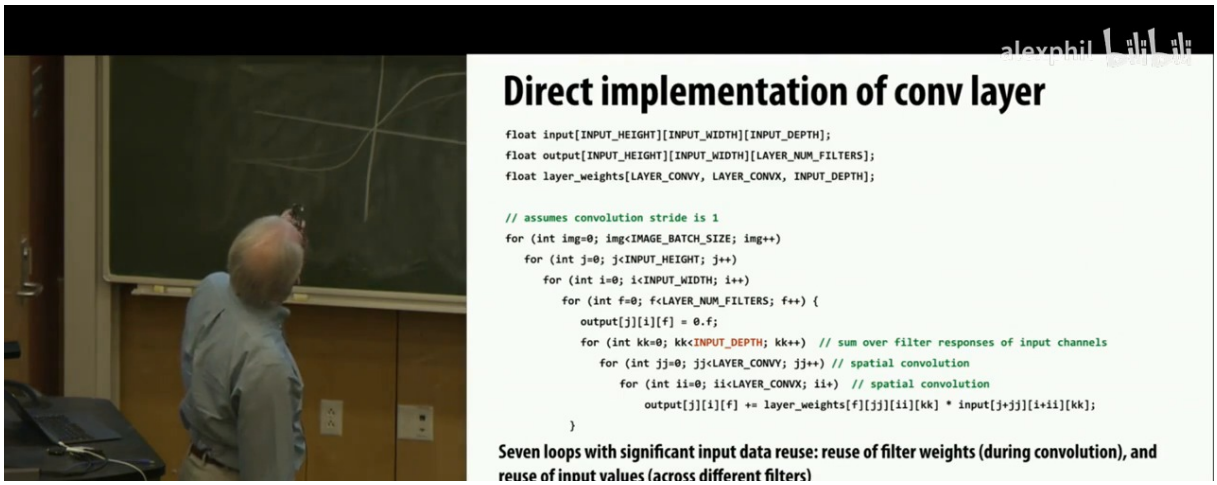


图 15: 卷积层的直接实现：多层嵌套循环[15]

3.5 池化

常见做法是某种池化：比如把四个像素平均成一个。但更常用的是最大池化——取四者中的最大值让它通过。

池化的作用是把”位置”信息换成”存在”信息

最大池化保留区域内最强的响应，丢弃其精确位置。这让网络对小的平移不敏感，从而获得一定的不变性。代价是空间分辨率下降，因此需要精确定位的任务不能过度池化。

讲者说这同样是数据缩减方案：你只想把最重要的东西推过去。

他展示了一个实际在用的图像网络：初始计算把 224×224 的图像用带步长的卷积降到 55×55，然后用 5×5 最大池化继续降，再用 3×3 最大池化继续降。末端用 softmax 给出分类。

他也指出：有些层是卷积的（权重在所有滤波器上相同），而稠密区域的那些则完全任意——每个权重对每个位置的每个副本都可以不同，因此可能的权重数量极其庞大。

3.6 本章小结

卷积层因权重共享而三重节省：更易训练、更易存储、更易评估。滤波器可以近似高斯模糊，也可以是边缘检测（求空间导数）。多滤波器是 k 个独立计算而非相互传递。纯线性无效，因此必须加非线性；池化做数据缩减，最大池化最常用。典型网络是”卷积 + ReLU + 池化”的序列，末端用 softmax 分类。

4 推理的计算结构

4.1 任务规模

讲者给出具体任务：给一张新的 $224 \times 224 \times 3$ 的 RGB 图像，如何算出那一百个输出值?

[15] 视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

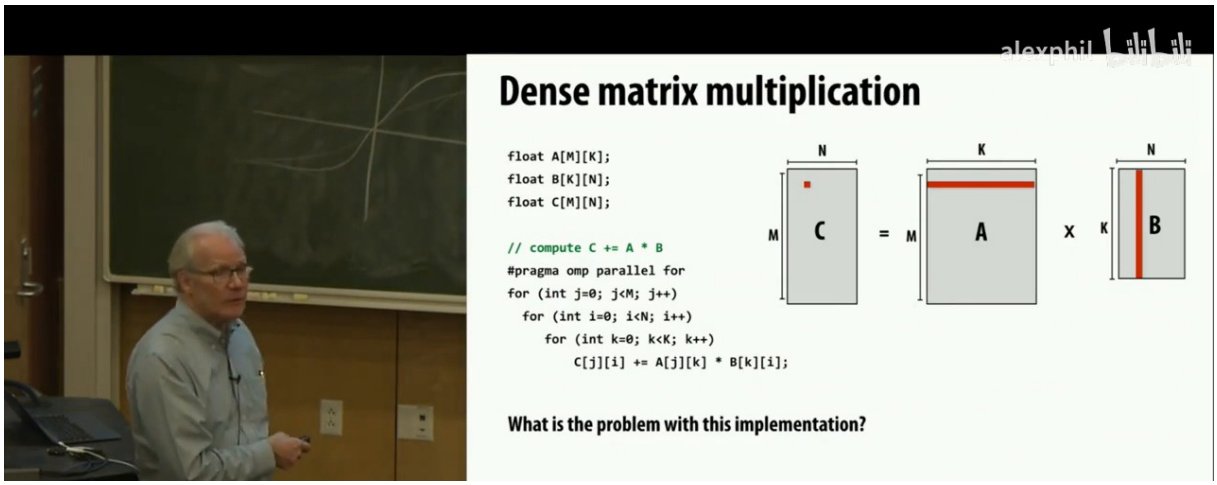


图 16: 稠密矩阵乘法：卷积与它在计算结构上相同[16]

一种方式是写很深的嵌套循环。以单个卷积层为例：外层遍历批中的每张图，再遍历输出的空间维度、滤波器数量与输入通道，最内层遍历卷积核的 x 与 y 维度。

最终就是大量乘加运算。

单层卷积的乘加次数可以估为：

$$\text{MACs} = H \cdot W \cdot K \cdot C_{\text{in}} \cdot k_h \cdot k_w$$

- H, W：输出特征图的空间尺寸；
- K：本层滤波器个数（输出通道数）；
- C_{in}：输入通道数，首层为 3（RGB）；
- k_h, k_w：卷积核的高与宽，3×3 时各为 3。

以 224×224 输入、3 通道、3×3 核为例，单层就已是百万量级的乘加，整网因此达十亿次以上。

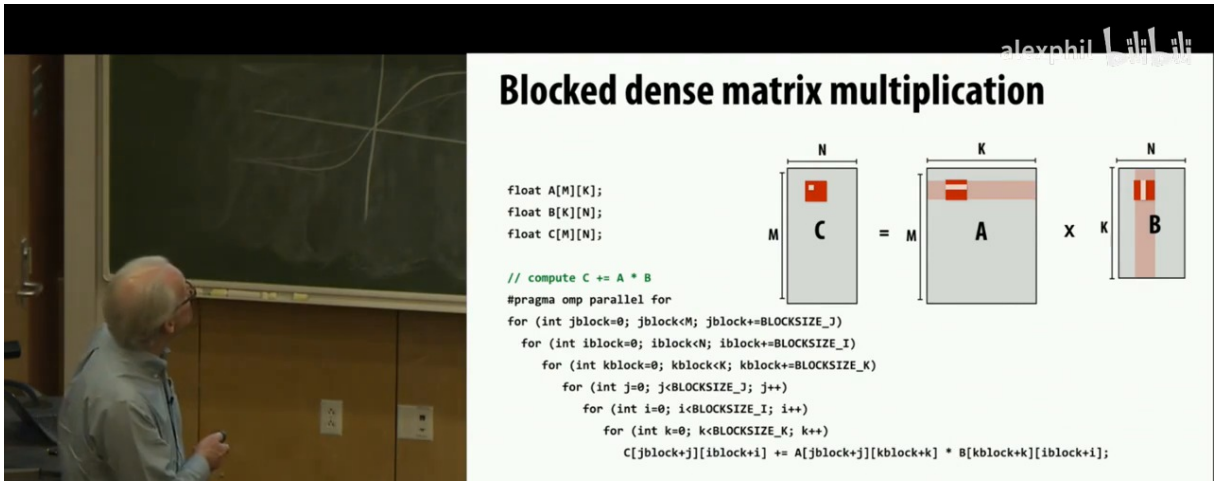


图 17: 分块稠密矩阵乘法：改善空间局部性[17]

[16] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

4.2 本章小结

卷积层因权重共享而三重节省：更易训练、存储与评估。滤波器可近似高斯模糊，也可做边缘检测（求空间导数）。多滤波器是 k 个独立计算而非相互传递。纯线性无效，因此必须加非线性；池化做数据缩减，最大池化最常用。典型网络是「卷积 + ReLU + 池化」的序列，末端用 softmax 分类。

有了网络结构，接下来的问题是：怎么把它算得快。

5 推理的计算结构

5.1 循环嵌套与局部性

讲者做了一段很有价值的分析：计算内层某个元素时，我们在 A 的所有行上跨步，这不算太糟，因为至少有空间局部性；而在 B 的所有列上跨步，这就很糟，因为没有空间局部性。

但两种情况下都没有任何时间局部性。因此缓存利用得很不好。

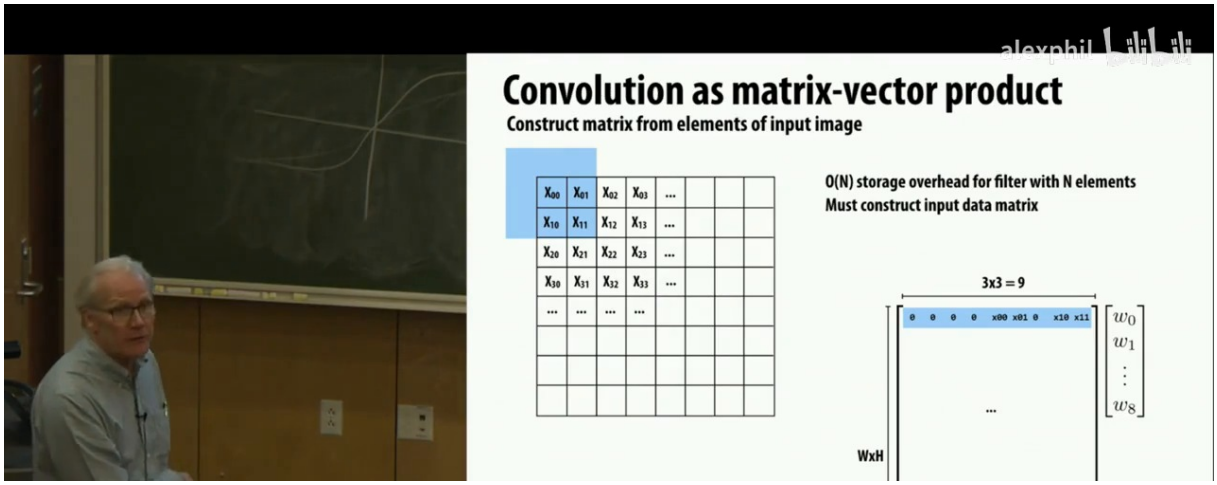


图 18: 把卷积写成矩阵向量积：从输入图像元素构造矩阵[18]

解法与前面各讲一致：引入分块以改善空间局部性——把 A 与 B 的某个子矩阵留住，在其元素上把计算做完再换。

[18] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

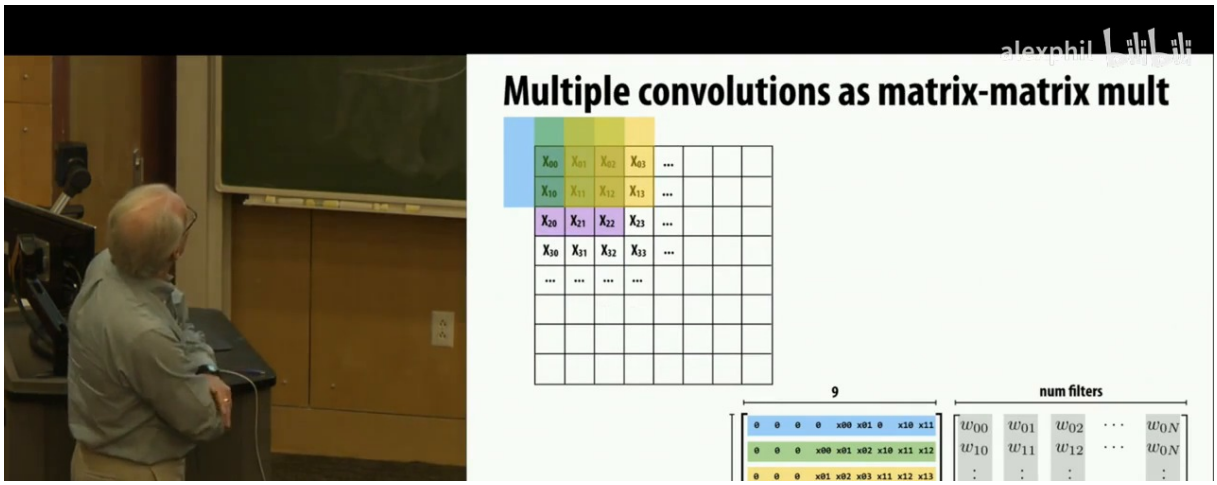


图 19: 多个卷积作为矩阵矩阵乘法[19]

卷积的优化与矩阵乘法同构

把卷积展开成矩阵乘后，所有针对 GEMM 的优化技术（分块、寄存器阻塞、向量化、多线程划分）都可以直接复用。这也是为什么深度学习框架的卷积实现往往调用高度优化的 GEMM 库，而不是自己写循环。

5.2 内存占用

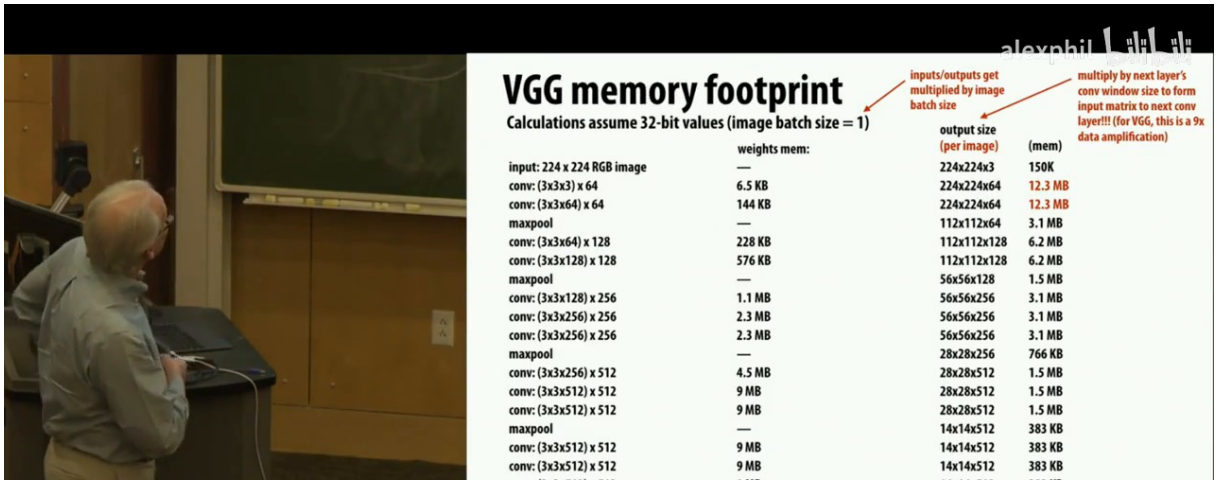


图 20: VGG 的内存占用：输入输出随层数逐级放大[20]

[19] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

[20] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

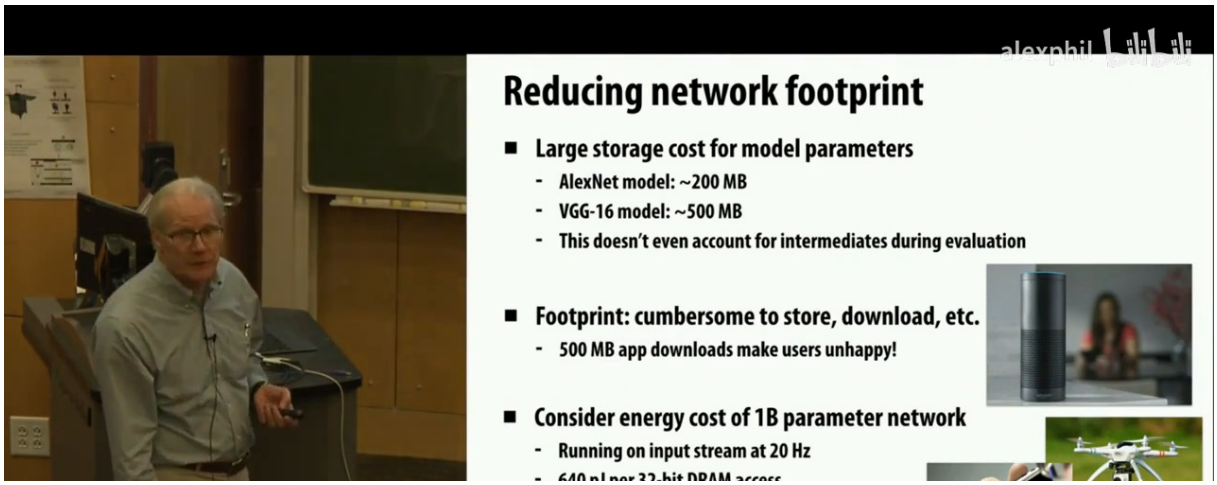


图 21: 降低网络内存占用：参数存储与中间结果[21]

讲者讨论了 VGG 的内存占用：输入与输出会逐层乘以滤波器数量与卷积窗口大小，因此中间结果的存储需求随层数快速增长。

参数量与算力是两个独立的约束

参数量决定模型能否装进片上存储、能否快速加载；算力决定单次推理需要多少周期。二者不一定同向变化：有些网络参数少但计算密，有些参数多但每参数只算一次。优化时必须先测出哪个是瓶颈，再决定压缩策略。

他给出具体数字：AlexNet 模型参数约 200 MB，而 VGG-16 更大。这说明推理的瓶颈可能不在算力而在内存带宽与容量。

推理的瓶颈常在内存而非算力

卷积的计算量大，但参数与中间结果的存储同样巨大。若每层都要把整个中间结果写回内存再读回，带宽就会成为限制。因此实际实现常做层融合（把相邻的卷积与 ReLU 合并）以避免中间结果的往返。

5.3 本章小结

推理任务是把 $224 \times 224 \times 3$ 的输入算成约一百个输出。朴素实现是多层嵌套循环，缓存行为差：B 的列遍历无空间局部性，且两者都无时间局部性。解法是分块，与矩阵乘法同构。内存占用随层数快速增长，AlexNet 参数约 200 MB，因此瓶颈常在带宽而非算力。

6 训练：梯度下降与损失

6.1 本章小结

推理任务是把 $224 \times 224 \times 3$ 的输入算成约一百个输出。朴素实现是多层嵌套循环，缓存行为差：B 的列遍历无空间局部性，且两者都无时间局部性。解法是分块，与矩阵乘法同构。内存占用随层数快速增长，AlexNet 参数约 200 MB，因此瓶颈常在带宽而非算力。

[21] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

推理只需前向计算。训练则要额外完成误差回传与参数更新，是更重的任务。

7　训练：梯度下降与损失

7.1　泛化与评估集

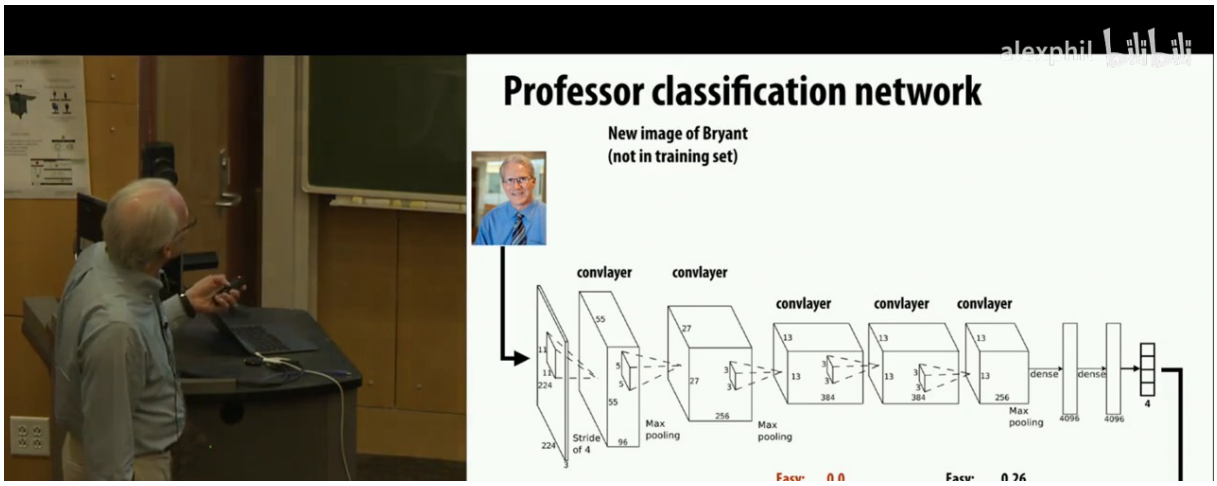


图 22: 训练循环：前向传播、计算损失、反向传播更新权重[22]

讲者说明训练要做什么：我们已经知道网络对该图像的输出，也知道它应该预测什么，现在想回去把这些权重稍微调整一点，让它变得更好——但不要改得太剧烈。

他指出了一个关键的张力：对某一张图像而言，让它在这一个样本上变得完美是相对容易的。但我们希望在整个巨大样本集上都能给出普遍良好的结果。

更重要的是，我们要它不仅能对训练过的样本工作，而且在丢进其他已知或未标注的图像时也能给出正确的值。

因此常见做法是把原始训练数据切出一部分，纯用于评估：在一个集合上训练，然后在另一个来自统计上相似分布的集合上评估质量。

泛化的代价决定了训练必须”小步走”

如果为单张图大幅调整权重，训练集上的误差会迅速下降，但模型会失去泛化能力。因此训练必须用很小的步长在大量样本上累积调整。这个约束解释了为什么训练需要极多轮迭代与极大数据量，也就解释了为什么训练的算力需求远高于推理。

[22] 视频画面时间区间：00:40:00–00:40:15。若为裁剪图，时间区间仍对应原视频画面。

7.2 softmax 与损失

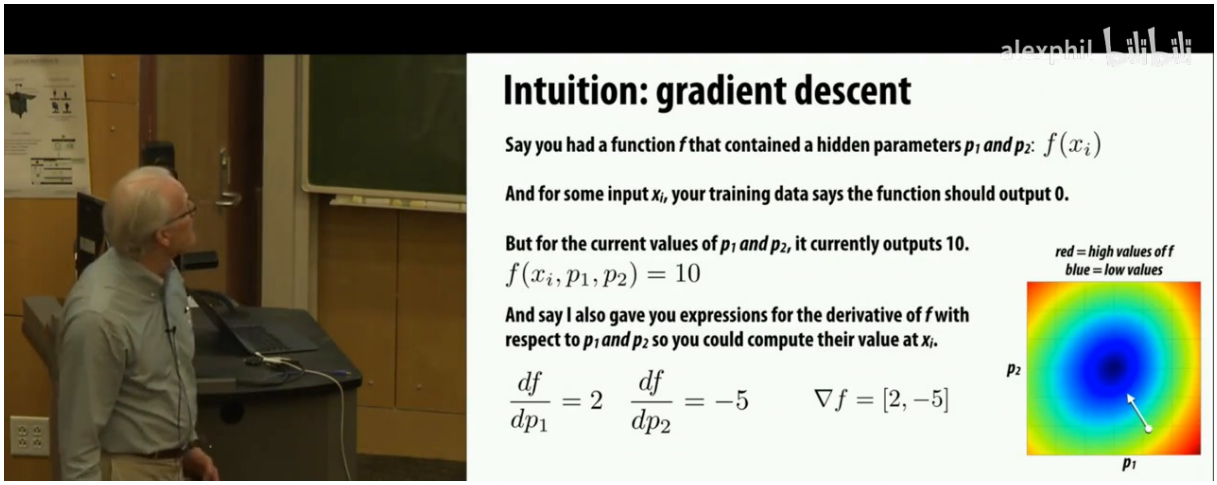


图 23: 梯度下降的直觉：沿函数下降方向调整参数[23]

softmax 的可微性是训练可行的前提

若直接取最大值，该操作在绝大多数点上梯度为零，参数无法通过梯度下降更新。softmax 用一个平滑的归一化替代硬选择，使梯度处处存在且能反映”次优类别与最优类别的差距”，这是反向传播能工作的关键。

讲者说明输出处理：若有若干实数输出，直接取最大值也算合理。但通常会让它们经过一个叫 softmax 的算子——它会强调最大的那个数，但不会把其余的全丢掉。

这样做部分是为了让函数可微，从而获得比简单取最大值更细的分辨率。

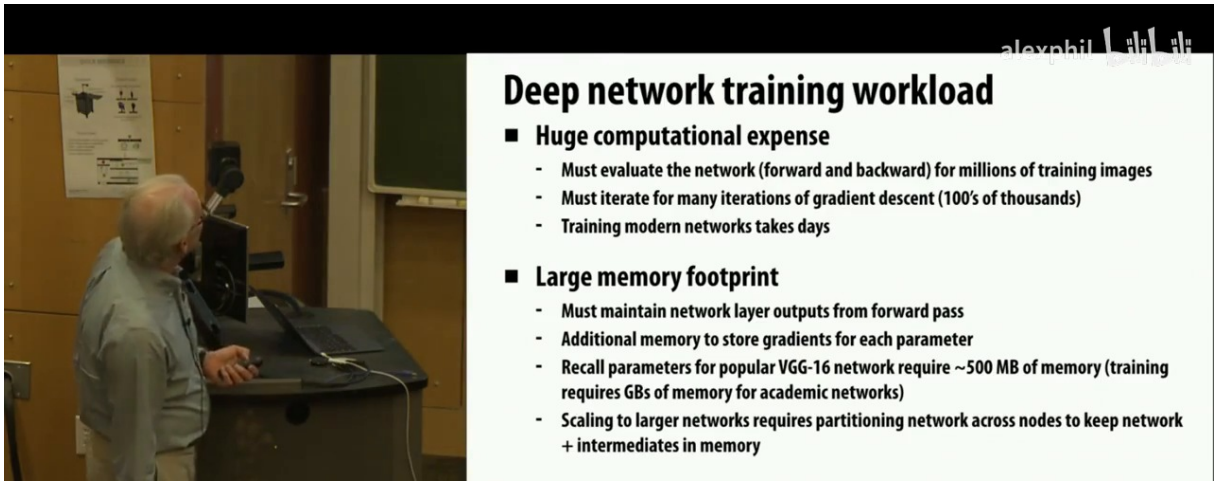


图 24: 深度网络的训练负载：前向与反向传播的巨大计算量[24]

于是损失就是预测值与目标值之间差异的某个函数，训练的目标是最小化这个损失。

$$L = -\sum_{c=1}^{C} y_c \log(\hat{y}_c), \qquad \hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$$

[23] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

[24] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

- z_c：第 c 类的原始输出分数；
- $\hat{y}_c$：经 softmax 归一化后的概率；
- y_c：真实标签的独热编码；
- C：类别数，本例约 100。

讲者强调 softmax 的可微性使梯度可以逐层回传，这正是反向传播得以实现的前提。

7.3 本章小结

训练的目标是通过小幅调整权重让网络在大量样本上普遍表现良好，并能泛化到未见样本；因此必须切出评估集衡量质量。输出经 softmax 强调最大值但不丢弃其余，使其可微从而支持梯度回传。损失衡量预测与目标的差异，训练即最小化该损失。

8 总结与延伸

讲者最后说讲了这么久，问有没有人熟悉这些内容，然后结束了这一讲。

把全讲串起来，这是一条从” 神经网络是什么” 到” 如何高效计算” 的路径。深度学习彻底改变了 AI，并把问题拉回结构化数值计算，这是它与并行计算天然契合的原因。基本单元是加权和加偏置再加非线性激活；早期用饱和型函数，现在普遍用 ReLU——取与零的最大值，简单却有效，且只是受大脑启发而非模仿。训练与推理是两个不同的问题：训练需要数天到数月与巨量算力，推理单次快速但调用次数极多且常在受限设备上进行。卷积层因权重共享而三重节省：更易训练、存储与评估；滤波器可以是高斯模糊，也可以是边缘检测；多滤波器是 k 个独立计算；纯线性无效，因此必须加非线性；池化做数据缩减。推理的朴素实现缓存行为差，解法是分块，与矩阵乘法同构；内存占用随层数快速增长，因此瓶颈常在带宽而非算力。训练的目标是泛化而非拟合单样本，输出经 softmax 使其可微，训练即最小化损失。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 深度学习的并行性来自它的数值结构，而非它的” 智能”。它由规则化的乘加构成，因此可以映射到 SIMD、GPU 与专用加速器。这也解释了为什么它对硬件演进极其敏感——算力提升直接转化为模型能力。
- 卷积的价值是参数效率，不是表达力。它用权重共享把参数量从” 与图像尺寸的乘积成正比” 降到” 只与滤波器大小有关”。因此它适用于具有平移不变性的任务，不适合所有任务。
- 推理与训练需要不同的优化策略。推理关注单次延迟与内存带宽，训练关注吞吐与数值稳定性（前向与反向都要算）。因此同一网络在两侧的最优实现往往不同。
- 非线性不是可选项而是必要条件。若全网络都是线性变换，多层叠加等价于单层线性变换，深度毫无意义。ReLU 的价值在于它足够简单，使梯度计算廉价，同时打破线性。

这堂课的边界在于它是导论：讲者明确说网络结构的设计仍带” 黑魔法” 成分，未深入反向传播的推导与具体训练算法细节。

两个可以直接从本讲数据提出的下一步问题：其一，卷积的权重共享依赖平移不变性，那么对于不具备这种不变性的任务（如需要精确定位的检测），应当如何改造卷积层？其二，推理的瓶颈常在内存带宽，那么把相邻层融合以减少中间结果往返，与把网络做窄以减少参数量，这两种策略在什么条件下更有效？